

Embedded Systems Engineering Roadmap

This roadmap contains more than enough resources to help you go as deep as you want into Embedded Systems Engineering.

Disclaimer:

You're **not expected to cover everything** mentioned here. Everyone's path is different, and it's okay to move at your own pace.

I can't specify exactly what you should focus on; **you may need additional resources not listed here.**

Give yourself room to make mistakes (just try not to make expensive ones). The important thing is to **keep learning and stay curious.**

Before diving into advanced topics, it's very important to build a strong foundation, starting with **mathematics and calculus**, followed by **electric circuits, electronics fundamentals**, and **computer architecture**.

These may not seem immediately necessary, but starting from the basics helps a lot. When you face challenges later, it won't be as overwhelming, and you'll have a solid base to rely on.

When starting, you might feel tempted to **skip steps** just to see quick results, that's normal. But try not to jump too far ahead. Embedded systems build on layers:

1. Circuits
2. Electronics
3. Then Embedded Systems

What Are Embedded Systems?

An embedded system is simply a design where a **microcontroller** is programmed to perform a **specific task**.

It combines **electronics, programming, and circuit design** to make a device that does one job efficiently.

When these systems communicate over the internet, we call that the **Internet of Things (IoT)**.

Applications can be found everywhere; in **medicine, power systems, consumer electronics, automotive, and many more.**

How the Learning Path Flows

1. Mathematics & Calculus

- You don't have to start from scratch if you're already comfortable with math, but a quick review helps.
- Knowing some basic **calculus** will also help later, Calculus (especially Differential Equations) is primarily needed for **Control Theory** and **Signal Processing** (both advanced topics listed later). For a typical beginner embedded project, basic algebra is sufficient.

2. Electric Circuits

- Study circuit analysis (Ohm's Law, KCL, KVL, Nodal & Mesh Analysis, Voltage Divider, etc.).
- Keep at least one good reference book for circuits (Art of Electronics is a good general reference, but for circuits analysis, many recommend Sadiku's Book)

3. PCB Design (Optional for Beginners)

- You can skip this part for now and return later when you're more comfortable.
- Once you understand circuits well, PCB design becomes easier.

4. Digital Design & Computer Architecture (Optional)

- Not mandatory at the start, but a deep understanding of **Digital Design** (logic gates, sequential logic) and **Computer Architecture** (memory, registers, pipelines) is necessary to move from "embedded programmer" to "embedded engineer."
- These topics give you a deeper understanding of how embedded systems work at the processor level.

5. Prototyping & Hardware Skills

- Learn how to use a **breadboard**.
- Practice **soldering** when needed.
- You can come back to these skills at any time.

6. Programming Fundamentals

- Start with the basics: **algorithms**, **data structures**, and simple logic.
- **C** is the main language for embedded systems.
- Later, you can explore **C++**, **Rust**, **Zig**, **Python/MicroPython** (used with Raspberry Pi), and **Assembly language** (for understanding low-level operations).

7. Microcontrollers

- Begin with **Arduino** — it's beginner-friendly and has a large community.
- If you can't get physical hardware, use [TinkerCAD](#), a free online Arduino simulator (there are others as well).
- As you advance, explore other microcontrollers (e.g., **STM32** – Professional/Industry, **ESP32**- IoT, **Raspberry Pi** mini-computer and probably a few others).
- Learn key concepts like **GPIO**, **ADC**, **Timers**, **PWM**, and **Watchdogs**.

8. Communication Protocols

- Understand how microcontrollers communicate:
 - **UART**
 - **I²C**
 - **SPI**
- Later, you can explore **wireless**, **industrial**, or **automotive** communication protocols.

9. Sensors and Actuators

- Sensors help the system interact with the environment (input).
- Actuators perform actions based on that data (output).

10. Operating Systems

- Embedded systems can be:
 - **Bare Metal** (programming that directly controls the hardware without an intervening OS layer)
 - **OS-based**, such as those using **Embedded Linux** (like Raspberry Pi).
- Learning the basics of operating systems is important for complex projects.
- **RTOS** (Real-Time Operating Systems)

11. Testing and Validation

- **Unit testing:** Learn to use frameworks (like Ceedling or Google Test for C) to test individual functions and modules of your code in isolation. These tests are often run on the host PC first.
- **Integration Testing:** Verify that different software modules and hardware peripherals work correctly when combined.
- **Hardware-in-the-Loop (HIL) Simulation:** Understand how to use specialized test setups to simulate the real-world environment and inputs/outputs, allowing for rigorous and repeatable testing of the entire embedded system.
- **Test-Driven Development (TDD):** Explore this methodology where you write tests *before* writing the actual functional code, ensuring your design is robust and verifiable from the start.

12. More Technical Areas

- Build System
- Debugging
- And many areas you'll discover as you progress.

13. Advanced & Specialized Areas

- Once you're confident, you can explore:
 - **Machine Learning / AI on embedded devices**
 - **FPGA design**
 - **Control theory**
 - **Simulation tools**
 - **Embedded Security**

14. Tools/IDE

- A **Development Environment** (e.g., VS Code, PlatformIO, STM32CubeIDE)
 - **Version Control (Git)** (I should have mentioned this earlier as these are mandatory industry skills).
-

About the GitHub Repository

This roadmap and its resources were compiled from a **GitHub repository** I'm currently using. It's structured in a simple way:

- **Introduction & Roadmap** – Explains the general flow
- **Basic Projects** – Starter projects to help you apply what you learn
- **Resources** –
 - Books (highly recommended)
 - Coursera, edX, and YouTube playlists
 - Articles and project guides

Some notable YouTube channels include [DigiKey](#), [Ben Eater](#), [Element14 Presents](#), and [Phil's Lab](#) — they're all great.

You'll also see icons to guide you:

-  **Baby icon** – beginner-level
-  **Book icon** – reference materials
-  **Diamond icon** – highly recommended resources (don't skip these!)

Each section contains materials in different formats — videos, books, or courses — so you can pick whichever works best for your learning style.

Here's the [LINK TO THE REPOSITORY](#)

Some notable mentions (do check their profiles out — they share great insights and resources):

1. [Balajee Seshadri](#)
2. [Rosmianto Aji Saputro](#)

Tips:

- **Build, Don't Just Read:** Projects go a long way in strengthening your understanding, so don't just read; build, test, collaborate, and explore. Hands-on experience teaches you faster.
- **Document Everything:** Documentation is as important as the project itself. How can you prove you did it? A video explaining the project / a demo showing what it does is better off a simple paragraph or heading. A good rule of thumb is to maintain a portfolio containing:
 - A short write up of each project
 - Screenshots, diagrams, or design files
 - A short video walkthrough or demoThis shows your depth of understanding and makes you stand out.

- **Study the Details (Datasheets, Industry Designs, and Trends):** These resources help you understand how professionals approach design challenges, optimize systems and ensure reliability. It also helps you stay up to date with evolving tools and protocols.
- **Certifications and Courses:** Certifications can be useful, but they should complement, not replace practical experience. Focus first on mastering fundamentals and building projects.

If you're exploring certifications, **Coursera** and **Udemy** have great options. Some recognized ones include:

- ARM Accredited Engineer
 - AWS IoT Certification
 - Microchip Advanced Engineering
- **Hardware is Competitive, Stand Out:** Build a strong foundation in electronics and embedded systems. Gain hands-on experience with debugging, prototyping, and validation, and understand manufacturing processes and cost optimization will help you stand out.
- Most importantly, **reach out when you need help.** Learning is always easier when you connect and grow with others.
Feel free to connect with me on [LinkedIn](#) if you'd like to contribute, collaborate, or learn together.
- **KEEP LEARNING.**

— *Compiled by Olonade Kelvin.*